intuitive

# BUSINESS REQUIREMENTS DOCUMENT

## Compliance Agent

| Field | Value |
|---|---|
| Document ID | BRD-CAG-001 |
| Version | 1.0 |
| Date | 22 October 2026 |
| Author | Pratham |
| Status | Draft — pending stakeholder review |
| Scope of this document | Phase 1 baseline architecture, core Obligation Engine, REST API, escalation scheduler, mock connectors, and forward-looking integration seams for Phases 2 and 3 |
| Target platforms | Python 3.11+ / FastAPI runtime, PostgreSQL 16 database, Docker / Docker Compose container environment, modern web browsers (vanilla HTML/JS dashboard) |
| Reference | [Reference/supersedes — none for v1.0] |

## Top Five Features

| # | Feature | The question it answers | Section |
|---|---|---|---|
| 1 | Unified Obligation Lifecycle Register | "Can we track every mandate—whether a policy acknowledgment, technical certification, or mandatory meeting—in one central ledger with a consistent status?" | Section 4.1 |
| 2 | Verifiable Evidence Recording | "Do we have tangible, auditable proof that an employee actually completed this specific obligation before marking them compliant?" | Section 4.1 |

intuitive

| # | Feature | The question it answers | Section |
|---|---|---|---|
| 3 | Automated Multi-Tier Escalation Scheduler | "Who is currently overdue, and did the system automatically chase the assignee and their manager on the designated schedule without human intervention?" | Section 4.2 |
| 4 | Teams & Meeting Attendance Synchronizer | "Did required staff actually show up to mandatory operational or regulatory sessions, and are no-shows flagged immediately?" | Section 4.3 |
| 5 | Cross-Organizational Scoreboard & Visibility | "What percentage of the workforce is fully compliant right now, broken down by obligation type and manager hierarchy?" | Section 4.4 |

## 1. Purpose

Enterprise obligations—such as annual policy acknowledgments, mandatory technical certifications, operational directives, statutory regulatory filings, and critical meeting attendances—are currently tracked across disconnected spreadsheets, ticketing systems, and departmental silos. Each mechanism operates with isolated reminder schedules, informal follow-ups, and fragmented completion records. Because no central system of record exists, leadership and compliance officers cannot obtain an authoritative, real-time posture of who is compliant, who is overdue, and who has already received formal escalation notices. Manual chasing by program owners is labor-intensive and inconsistently applied, while proof of completion remains dispersed across ephemeral email threads and ad-hoc meeting export files.

This document establishes the business, functional, and architectural requirements for the Compliance Agent. Built as a phased platform, the system addresses these gaps by delivering a single, generic Obligation Engine where all obligation types are processed through a unified database model and standard REST API. This document details the completed Phase 1 baseline—including the core lifecycle engine, multi-tier escalation scheduler, evidence capture mechanisms, and decoupled connector framework—while defining the architectural boundaries, risks, and functional roadmaps for Phase 2 (enterprise HRIS and Microsoft Graph integrations) and Phase 3 (cross-obligation defaulter scrutiny).

intuitive

## 2. Objectives

| ID | Objective | Success measure |
|---|---|---|
| OBJ-01 | One register for every obligation type | All six types (policy_ack, training, certification, directive, regulatory, meeting_attendance) are stored and queried through the same API with no per-type code paths. |
| OBJ-02 | Proof of compliance on record | Each obligation flips to compliant only when evidence is recorded; evidence type and raw reference are kept. |
| OBJ-03 | Automatic, repeatable escalation | Overdue obligations move through the ladder (day 3 / 7 / 14 / 21 by default) with each step fired once; repeated sweeps create no duplicate events. |
| OBJ-04 | Live visibility of compliance standing | Scoreboard counts by status, filterable by type and assignee; dashboard shows own status, team status and items needing attention. |
| OBJ-05 | Swap mocks for real systems without rework | Replacing MockEmailProvider or the mock Teams provider changes only the provider class; engine, API and data model are untouched. |

## 3. Scope

### 3.1 In scope

- Single relational `obligations` table supported by `obligation_evidence` and `escalation_events` child tables.
- Core REST API endpoints providing obligation creation, listing/filtering, detail retrieval, evidence recording, `/scoreboard`, `/directory`, and `/health`.

intuitive

- Automated escalation engine supporting configurable per-type reminder ladders, executed via an independent APScheduler daemon process.
- Email acknowledgment connector with an abstracted interface, exercised via an in-memory mock provider.
- Microsoft Teams attendance connector supporting single and recurring meetings, distinct handling of required versus optional attendees, and an in-memory mock provider.
- In-memory stub organizational directory containing 7 user entities and hierarchical manager relationship links.
- Same-origin, zero-build dashboard served at `` `/ui` `` displaying compliance scoreboards, active obligations, meeting attendance breakdowns, team status views, obligation authoring, and evidence submission.
- Alembic database migrations and complete pytest test suite validated against an isolated PostgreSQL instance.

## 3.2 Explicitly deferred

- HRIS integration and real manager resolution (Deferred to Phase 2; stub directory stands in and escalations go to created_by).
- Real email provider (e.g. Microsoft Graph) and inbound reply detection (Mock proves the seam; simulate_employee_reply stands in for reply parsing).
- Real Teams provider (Graph onlineMeetings and attendanceReports) (Mock proves the seam).
- Authentication and a real 'current user' (Not in Phase 1; dashboard has no login).
- Defaulter-scrutiny agent (patterns of repeated misses) (Deferred to Phase 3; needs cross-obligation analysis).

*Note: These items are explicitly prioritized candidates for subsequent execution phases; refer to Section 10 (Future Scope) for implementation prerequisites.*

## 3.3 Permanently excluded (not a phasing decision)

- Implementation of per-obligation-type tables, distinct per-type API endpoints, or type-based conditional branching in the core data engine.
- Treating an optional meeting invitee skipping a meeting as a compliance failure.
- Processing backward-looking reminder ladders for meeting occurrences that have already concluded.

intuitive

*These exclusions are foundational architectural boundaries and governance decisions; they will not be incorporated into the platform regardless of phase.*

---

# 4. Detailed Requirements

## 4.1 Registry & Evidence Management

| ID | Requirement | Priority |
|---|---|---|
| BR-01 | Create an obligation of any type with source, title, description, issuer, assignee, deadline and an evidence-required flag. | M |
| BR-02 | List obligations, filtered by assignee, status and type, newest first. | M |
| BR-03 | Return an obligation with its evidence and escalation history. | M |
| BR-04 | Record evidence against an obligation and set it to compliant. | M |
| BR-07 | Exempt obligations with evidence_required=false from overdue and escalation. | M |

## 4.2 Escalation Engine & Scheduling

| ID | Requirement | Priority |
|---|---|---|
| BR-05 | Mark past-deadline obligations overdue and fire escalation-ladder steps once each; repeated sweeps never duplicate events. | M |
| BR-06 | Allow a per-type ladder override without touching the engine (meeting attendance: same-day notice to the organizer only). | M |
| BR-08 | Run the escalation sweep as a separate scheduled process on a configurable interval (default 3600 s). | M |

intuitive

## 4.3 Integration Connectors & Ingestion

| ID | Requirement | Priority |
|---|---|---|
| BR-09 | Send obligation notices by email and record acknowledgment evidence through the public API, never the database. | M |
| BR-10 | Turn each Teams meeting occurrence into one obligation per expected attendee, idempotently, and record join/leave evidence from the attendance report. | M |
| BR-14 | Real email and Teams providers behind the existing provider interfaces. | S |

## 4.4 Reporting, Directory & Advanced Analytics

| ID | Requirement | Priority |
|---|---|---|
| BR-11 | Scoreboard of counts by status, filterable by type and assignee, optionally grouped by assignee. | S |
| BR-12 | Dashboard showing own status, team status by person, needs-attention items, meetings by series and occurrence, and per-person detail. | S |
| BR-13 | Directory lookup of users and direct reports. | S |
| BR-15 | HRIS-backed manager resolution so escalations reach the real manager. | S |
| BR-16 | Authentication and a real current-user concept. | C |
| BR-17 | Defaulter-scrutiny agent that detects repeated misses across obligations. | C |

intuitive

# 5. Solution Approach

## 5.1 Process Flow

Obligations originate via direct API ingestion or connector-driven synchronization, entering the system in a pending state with defined deadlines and evidence requirements. External actors interact through dedicated channels—such as replying to notification emails or attending scheduled Teams conferences—which prompts the respective connector to extract completion artifacts and submit them via the standard REST evidence endpoint, transitioning the record to compliant. Concurrently, an autonomous scheduler process executes hourly database sweeps against open obligations; records that exceed their deadline without valid evidence transition to overdue and step progressively through a deterministic escalation ladder until the final tier marks them escalated, after which all aggregate metrics and audit histories stream directly into the unified web scoreboard and dashboard interfaces.

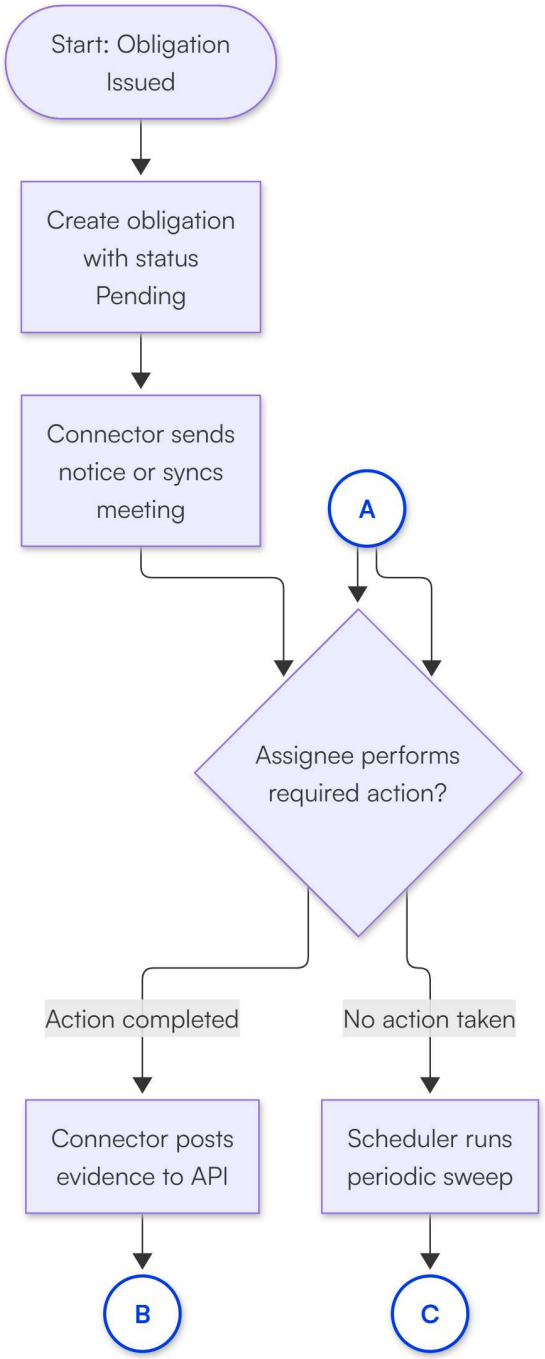



Figure 1 — Process Flow (part 1 of 3)

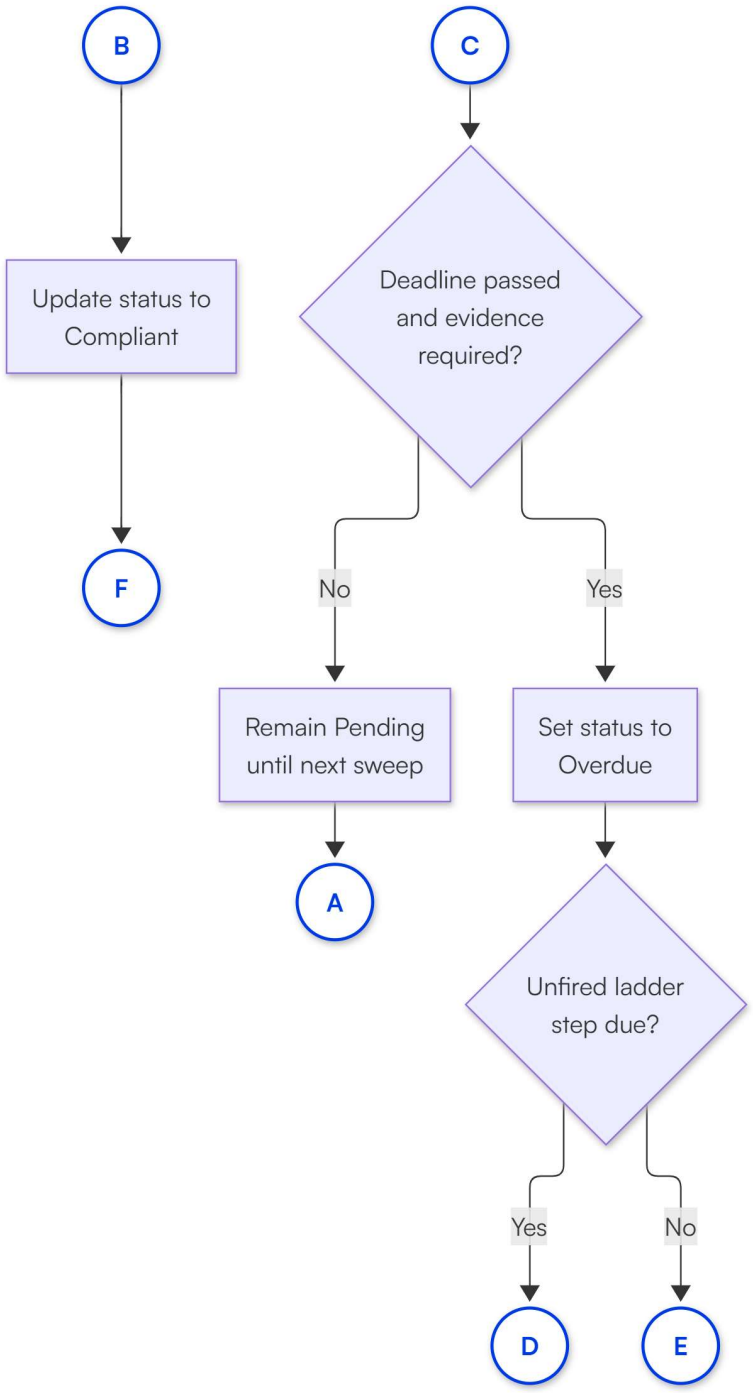



Figure 1 — Process Flow (part 2 of 3)

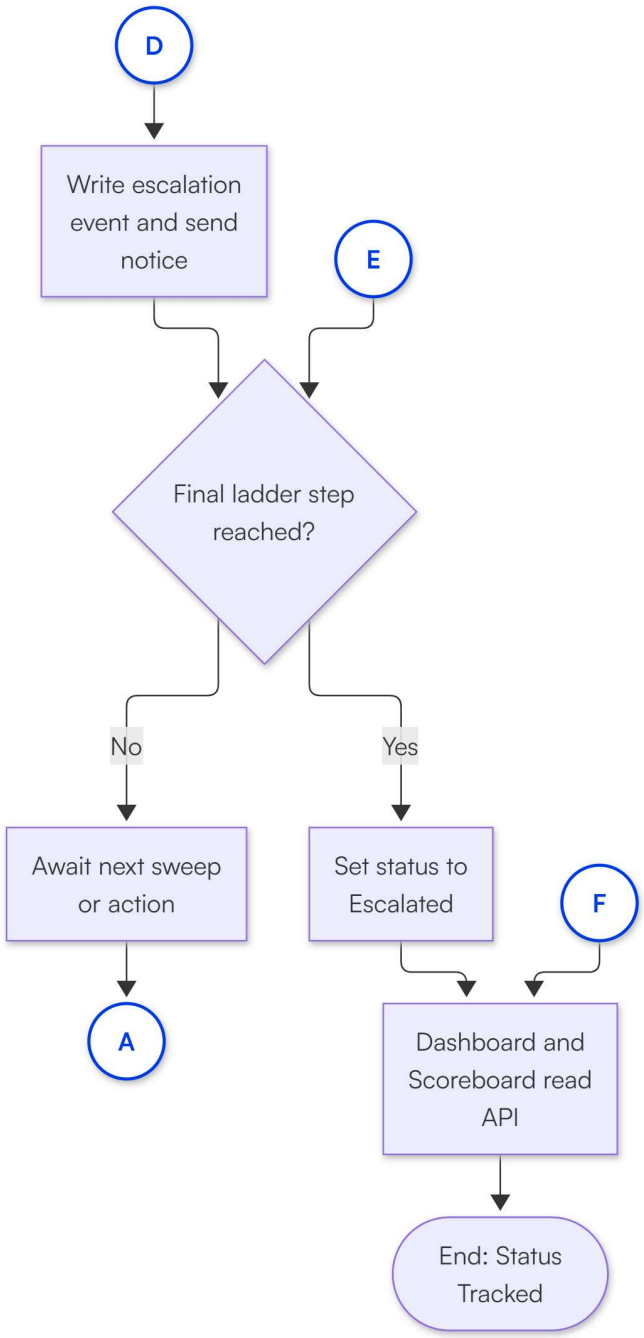



Figure 1 — Process Flow (part 3 of 3)

## 5.2 Architecture

The platform is structured into three isolated operational tiers—the FastAPI Application Service, the APScheduler Background Service, and specialized external connectors—all mediated by a common PostgreSQL relational database and standard HTTP contracts. Connectors reside strictly within their isolated package boundary and are prohibited from querying the database directly, interacting with the system exclusively through authenticated REST endpoints to register obligations and

submit evidence payload references. The escalation scheduler operates as a decoupled background worker querying non-compliant records, evaluating ladder eligibility, writing immutable escalation logs directly to the database, and issuing external alerts across abstracted provider interfaces without impeding synchronous API transactions.

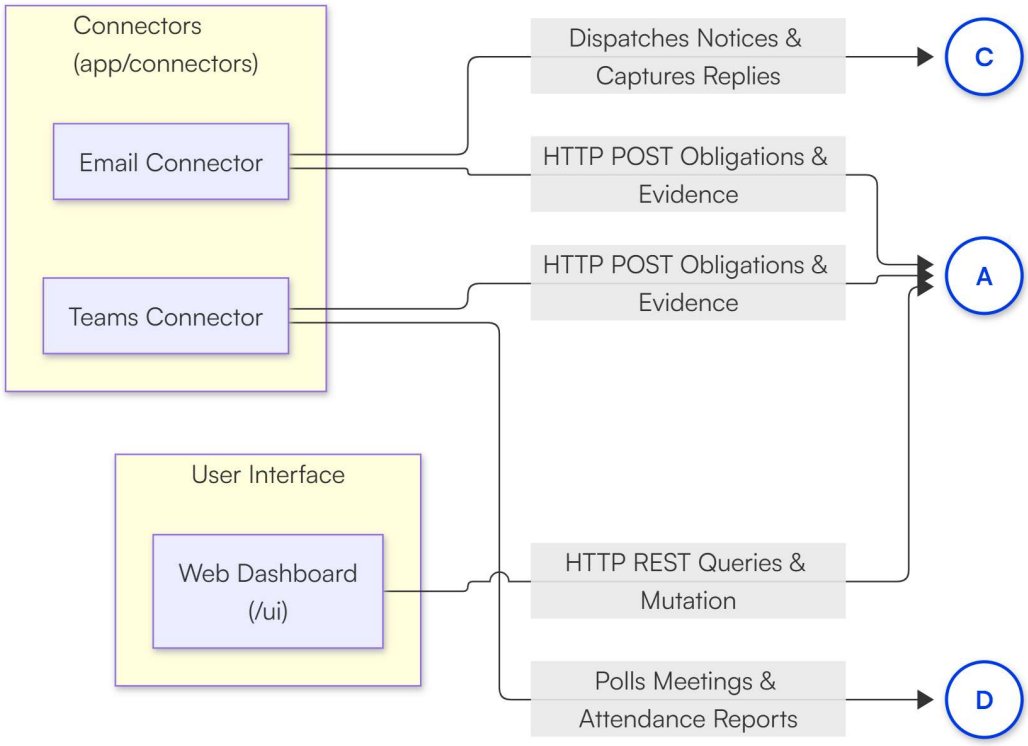



Figure 2 — Architecture (part 1 of 3)

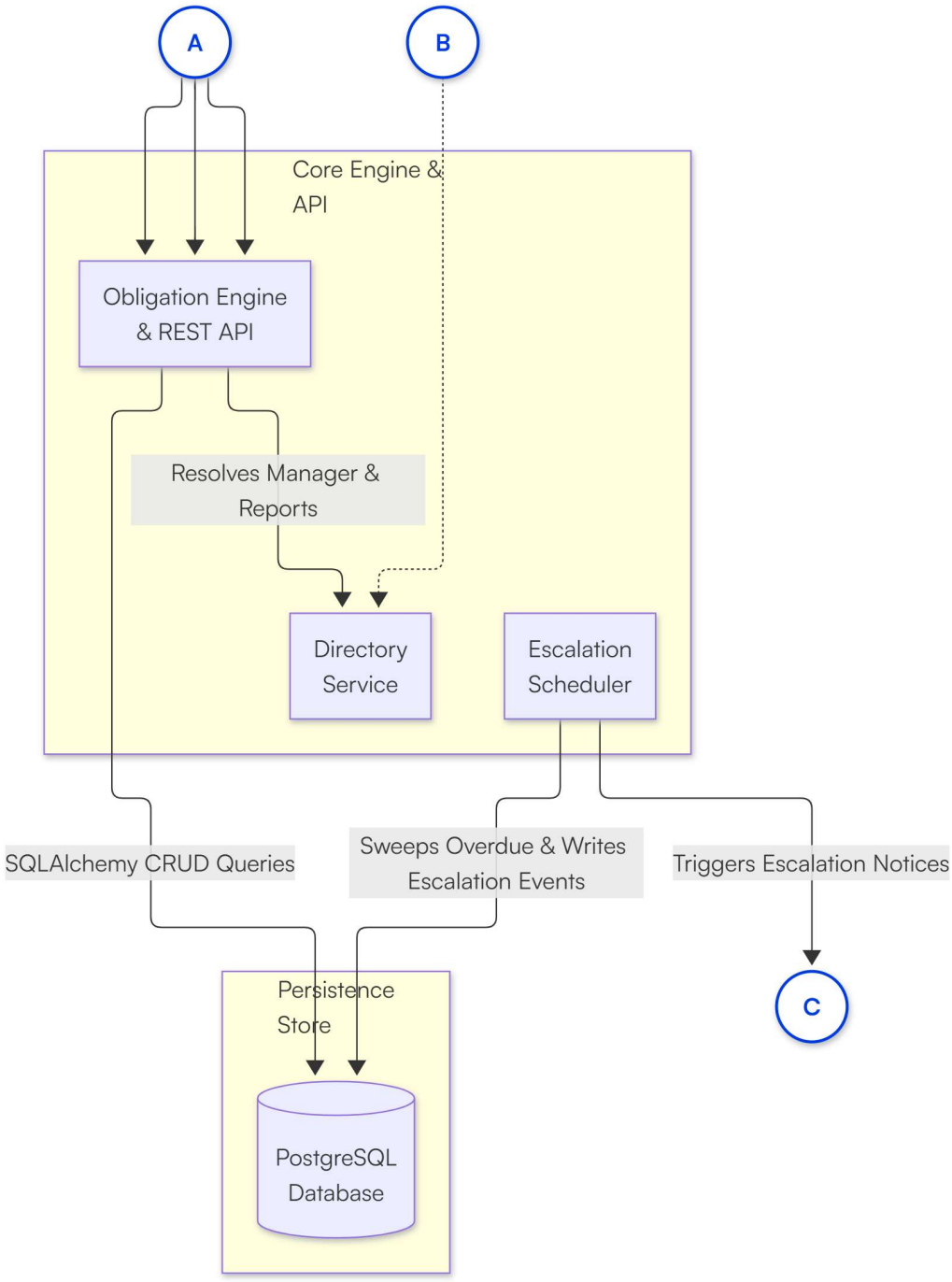



Figure 2 — Architecture (part 2 of 3)

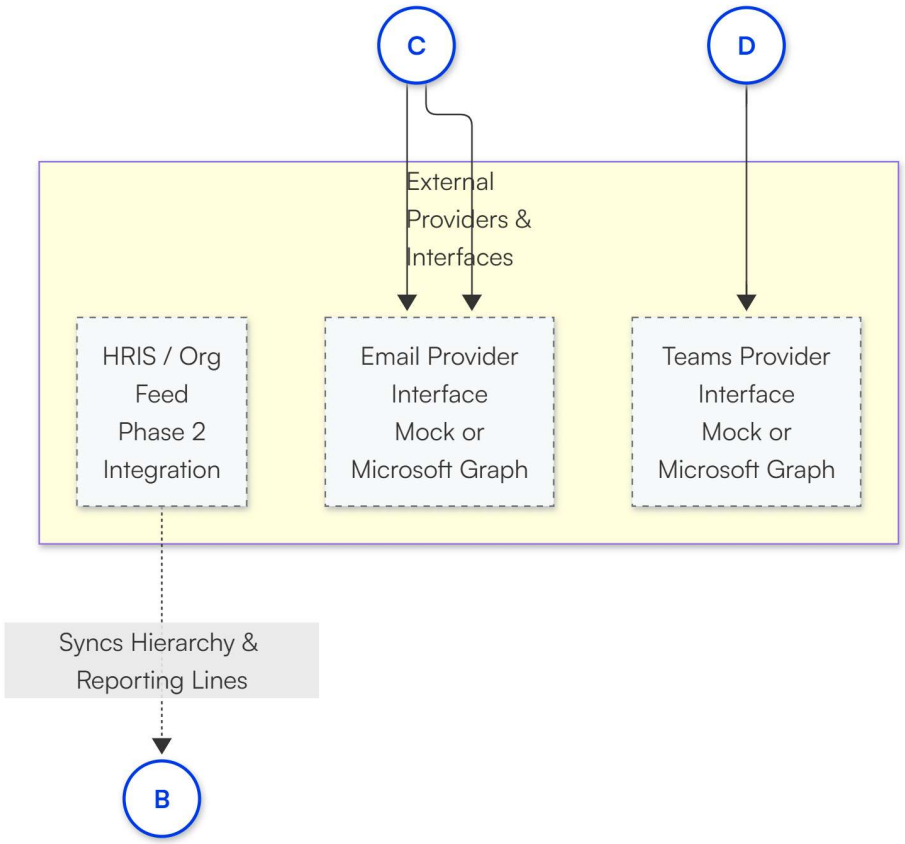



Figure 2 — Architecture (part 3 of 3)

## 5.3 Proposed Technology Stack

| Layer | Proposal | Notes |
|---|---|---|
| Language & Runtime | Python 3.11+ | Provides strong typing capabilities, native performance improvements, and ecosystem compatibility with modern asynchronous and data frameworks. |
| API Framework | FastAPI & Uvicorn | Offers high-throughput ASGI request processing, automatic OpenAPI specification generation, and native integration with Pydantic validation models. |

intuitive

| Layer | Proposal | Notes |
|---|---|---|
| Data Modeling & ORM | SQLAlchemy 2.0 (Sync) & Pydantic | Enforces strict schema validation, type safety, and standard 2.0 declarative syntax without the operational overhead of async drivers for batch workers. |
| Database Engine | PostgreSQL 16 (via Docker Compose) | Enterprise-grade relational engine supporting atomic transactions, robust indexing, foreign key cascade constraints, and native timestamp handling. |
| Schema Migrations | Alembic | Ensures repeatable, version-controlled database schema upgrades and automated drift detection across local and production environments. |
| Job Scheduling | APScheduler | Lightweight, in-process Python job scheduling that eliminates external broker dependencies (e.g., Celery, Redis) during initial operational phases. |
| HTTP Client | HTTPX | Robust synchronous and asynchronous HTTP client used by connectors to interact with the core REST API over network boundaries. |
| Test Framework | pytest & pytest-cov | Enables comprehensive integration and regression testing against an ephemeral, real PostgreSQL database instance per test session. |
| Presentation Layer | Vanilla HTML5 / Modern JavaScript | Zero-build, low-overhead browser dashboard served directly from the FastAPI static mount, eliminating front-end build pipelines. |

intuitive

## 5.4 Design Principles

- **Single Table Polymorphism:** Every obligation regardless of functional domain (training, policy, legal, meeting) is captured as a row in the single `obligations` table; per-type database schemas, dedicated endpoints, and conditional ORM subclasses are strictly forbidden.
- **Strict Connector Encapsulation:** Integration logic specific to third-party platforms (Microsoft Graph, HRIS, email) lives exclusively within `app/connectors/` and communicates with the system solely via the public HTTP REST API.
- **Evidence Precedes Compliance:** An obligation cannot transition to a compliant state through administrative override or heuristic inference; status transition requires a validated entry in the evidence sub-ledger.
- **Idempotent Sweep Execution:** The escalation sweep must be completely safe to execute arbitrarily often; an escalation tier already recorded for a given obligation cycle must never be re-fired or duplicated.
- **Decoupled Provider Boundaries:** Provider implementations (email dispatch, meeting attendance extraction) must sit behind rigid abstraction interfaces so mock adapters can be swapped for production enterprise APIs without modifying core application code.

---

# 6. Non-Functional Requirements

## 6.1 Idempotency

- The escalation sweep process must be safe to execute continuously without side effects; an already-fired ladder step for a given obligation must never produce duplicate events or repeat notifications.
- Teams meeting synchronization routines must be idempotent, matching existing database records by meeting occurrence ID to update records rather than generating redundant obligations.

## 6.2 Extensibility

- Registering a new obligation type must require only the addition of an enumeration value to the schema and, where applicable, a corresponding ladder configuration entry.

intuitive

- Integrating a new external transport or data source must require solely the concrete implementation of the defined provider interface without core engine modification.

## 6.3 Data Integrity

- All database timestamps across all tables must be stored as explicit, timezone-aware UTC values (`DateTime(timezone=True)`).
- Relational child records (`obligation_evidence`, `escalation_events`) must maintain foreign key integrity with cascade-delete constraints tied to the parent obligation record.
- Database columns frequently evaluated by the engine—specifically `status`, `type`, and `assigned_to`—must maintain dedicated indexes to sustain query efficiency.

## 6.4 Separation of Concerns

- The REST API, the background escalation scheduler, and the integration connectors must execute as distinct, isolated operating system processes.
- External integration connectors must interact with the application tier exclusively through external HTTP endpoints, with no direct access to the database layer.

## 6.5 Configurability

- System configuration parameters—including database connection strings, worker sweep intervals (defaulting to 3600 seconds), and environment profiles—must be externalized via environment variables and local `.env` files.
- System defaults must achieve out-of-the-box local execution using the standard bundled Docker Compose specification.

## 6.6 Observability

- The background scheduler must emit structured log entries capturing the total count of evaluated obligations and newly generated escalation events per sweep.
- Production deployments must implement structured JSON logging and automated alerting for unhandled sweep failures or missed execution intervals (proposed).

## 6.7 Testability

- The automated test harness must execute all integration test suites against an isolated, physical PostgreSQL database instance provisioned per test session.

intuitive

- Connector harnesses and the core escalation engine must maintain separate, dedicated test files achieving isolated component verification.

## 6.8 Security

- Phase 1 contains no authentication or authorization controls; local default database credentials must remain strictly confined to development and evaluation environments.
- Production deployment (Phase 2) requires enterprise identity integration, role-based access control (RBAC), and externalized secret storage out of source control (proposed).

## 6.9 Auditability

- Every escalation action executed by the platform must generate an immutable log record capturing the exact ladder step, UTC timestamp, and target recipient.
- Every status flip to compliant must be backed by a corresponding record in the evidence ledger detailing the evidence type and raw data reference.

## 6.10 Performance

- End-to-end background escalation sweep time must complete within an acceptable operational window, and API read/write latency must sustain typical enterprise loads (P95 targets to be formally benchmarked once production dataset volumes are established).

---

# 7. Risks

| ID | Risk | Impact | Mitigation |
|---|---|---|---|
| R-01 | No authentication: anyone who can reach the API can record evidence and flip an obligation to compliant. | High | Add authentication and role checks before any non-demo use; restrict the evidence endpoint to connectors and the assignee. |

intuitive

| ID | Risk | Impact | Mitigation |
|---|---|---|---|
| R-02 | Evidence is accepted unverified, so a bad or forged call produces false compliance. | High | Validate evidence by type (reply sender matches assignee, attendance matches occurrence); keep raw references for audit. |
| R-03 | The scheduler notifies through the in-memory mock, so no real notices go out. | High | Wire a real EmailProvider before launch; add a startup check that fails if the mock is active outside dev. |
| R-04 | Escalations go to created_by, not the employee's manager, so the wrong person may be chased. | Medium | Phase 2 HRIS manager resolution; until then document the rule and let issuers name an escalation contact. |
| R-05 | The email notice is sent before the event row is committed, so a mid-sweep failure can resend notices on the next run. | Medium | Commit each event before sending, or record send state per event; add retry with a dead-letter log. |
| R-06 | Single blocking scheduler process: if it stops, nothing escalates and nobody is told. | Medium | Run under a supervisor, log a heartbeat per sweep, and alert when a sweep is missed. |
| R-07 | The email connector keeps message-to-obligation links in memory; a restart loses the ability to match replies. | Medium | Persist the mapping (or use Message-ID and In-Reply-To headers) in the database. |
| R-08 | Teams sync finds existing obligations by title and deadline, which breaks if a title changes and slows as obligations grow. | Low | Key obligations on the provider's occurrence id and add a lookup index. |

intuitive

## 8. Delivery Phases

| Phase | Content | Exit Criteria |
|---|---|---|
| Phase 1: Obligation Engine (done) | Generic obligation model, evidence, escalation engine and scheduler, email and Teams connectors on mocks, scoreboard, stub directory, dashboard. | Test suite passes against Postgres; a mocked policy acknowledgment and a mocked Teams meeting each run from creation to compliant or escalated through the one API. |
| Phase 2: Real integrations | HRIS directory sync and manager resolution; real email and Teams providers via Microsoft Graph; inbound reply detection; authentication. | Escalations reach the real manager; a real reply or attendance report marks an obligation compliant; unauthenticated writes are refused. |
| Phase 3: Defaulter scrutiny | Agent that analyses misses across obligations and people, and flags repeat defaulters. | Repeat-miss patterns surface for a manager or compliance officer without manual queries; false-positive rate reviewed with the compliance owner. |
| Hardening (alongside Phases 2 and 3) | Fixes for the Risks table: durable email correlation, send-after-commit order, scheduler monitoring. | Each High and Medium risk has a closed mitigation or an accepted-risk note. |

*Phase 1 represents the foundational prerequisites for all subsequent milestones; Phase 2 unlocks external identity and communication channels, Phase 3 depends on cross-obligation historical data yielded by Phase 2, and Hardening activities execute concurrently across Phases 2 and 3.*

intuitive

## 9. Open Questions

- Who approves this document and the Phase 2 scope? The repo names no decision-maker.
- Are the day 3 / 7 / 14 / 21 ladder thresholds final? The code calls them illustrative.
- Should a regulatory obligation get a faster ladder than the default? The override slot exists and is empty.
- What should the escalation target be before HRIS arrives: created_by, or a named contact on each obligation?
- What counts as valid evidence for each type, and who may record it?
- Which identity system will provide authentication: Microsoft Entra ID or another?
- Is a retention period or audit-export requirement needed for evidence and escalation history?
- Where will it be hosted, and who operates the scheduler?
- The README points to PHASE1_CONTEXT.md (in a sibling claude-session folder), which was not available when drafting. Does it hold requirements that should replace anything here?

---

## 10. Future Scope (Deferred, Not Rejected)

*Notice: Items listed in this section are candidates for subsequent delivery phases; none of these features are committed, estimated, or approved for production release under the Phase 1 scope.*

| Item | What it would unlock | What it would require |
|---|---|---|
| HRIS integration and real manager resolution | Routing overdue escalations directly to an employee's actual reporting manager rather than the obligation issuer. | Production HRIS API integration (e.g., Workday, BambooHR), automated schema synchronization, and organizational hierarchy ingestion jobs. |

intuitive

| Item | What it would unlock | What it would require |
|---|---|---|
| Real email provider (Microsoft Graph) & reply detection | Autonomous end-to-end delivery of compliance notices and automated processing of employee inbound email acknowledgments. | Azure Entra ID app registration, Microsoft Graph Mail.Send/Mail.Read permissions, durable message header tracking, and webhook ingestion infrastructure. |
| Real Teams provider (Microsoft Graph onlineMeetings) | Automatic synchronization of live tenant meeting occurrences and parsing of actual participant join/leave attendance reports. | Microsoft Graph OnlineMeetings.Read and meeting attendance report application permissions, alongside occurrence reconciliation logic. |
| Enterprise authentication & current-user context | Secure API boundaries, tenant isolation, role-based UI access, and non-repudiable user action attribution. | OAuth2 / OIDC identity integration (e.g., Entra ID), JWT validation middleware, user session handling, and front-end login flows. |
| Defaulter-scrutiny agent | Proactive risk identification flagging individuals or teams displaying systemic patterns of overdue compliance across obligation categories. | Analytical query models across historical compliance timelines, anomaly detection rules, and compliance officer review dashboards. |

## 11. Data Model

Status values:

- `'pending'`



- `compliant`
- `overdue`
- `escalated`

Escalation steps:

- `reminder_1`
- `reminder_2`
- `escalation_manager`
- `escalation_leadership`

Tables:

- **obligations**: `id`, `type`, `source`, `title`, `description`, `created_by`, `assigned_to`, `deadline`, `evidence_required`, `status`, `created_at`, `updated_at`
- **obligation_evidence**: `id`, `obligation_id`, `evidence_type`, `recorded_at`, `raw_reference`
- **escalation_events**: `id`, `obligation_id`, `step`, `triggered_at`, `recipient`

---

## 12. Escalation Ladders

Default ladder (all types except meetings):

- **Day 3**: `reminder_1` -> Assignee
- **Day 7**: `reminder_2` -> Assignee
- **Day 14**: `escalation_manager` -> Issuer (`created_by`)
- **Day 21**: `escalation_leadership` -> Issuer; status becomes `escalated`

Meeting attendance ladder:

- **Day 0**: `escalation_manager` -> Organizer (same day notification, no leadership step)

intuitive

## 13. API Surface

REST endpoints:

- `POST /obligations`: Create an obligation of any type
- `GET /obligations`: List, filtered by `assigned_to`, `status`, `type`
- `GET /obligations/{id}`: Detail with evidence and escalation events
- `POST /obligations/{id}/evidence`: Record evidence; sets status to compliant
- `GET /scoreboard`: Counts by status; filter by `type`, `assigned_to`; `group_by=assigned_to`
- `GET /directory/users`, `/directory/users/{email}`: Stub directory and direct reports
- `GET /health`: Liveness check
- `/ui`: Same-origin dashboard

## 14. Sign-Off

| Role | Name | Signature | Date |
|---|---|---|---|
| Executive Sponsor | [TBD] | _____ | [Date] |
| Compliance Program Sponsor | [TBD] | _____ | [Date] |
| Engineering & IT Platform Lead | [TBD] | _____ | [Date] |